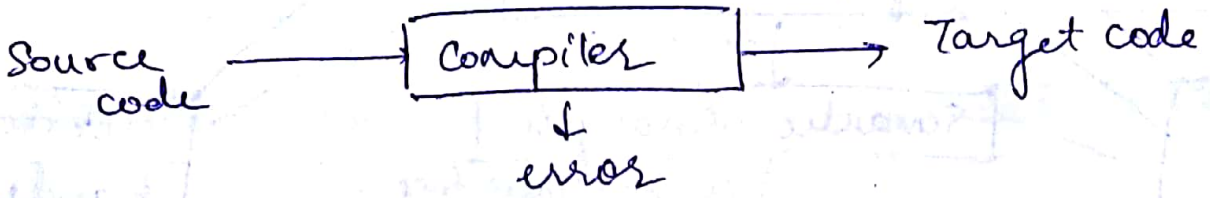
Unit-1

☆ Compilers

☆ Code → Target code
Lang. 1 Lang 2.

☆ reports errors.



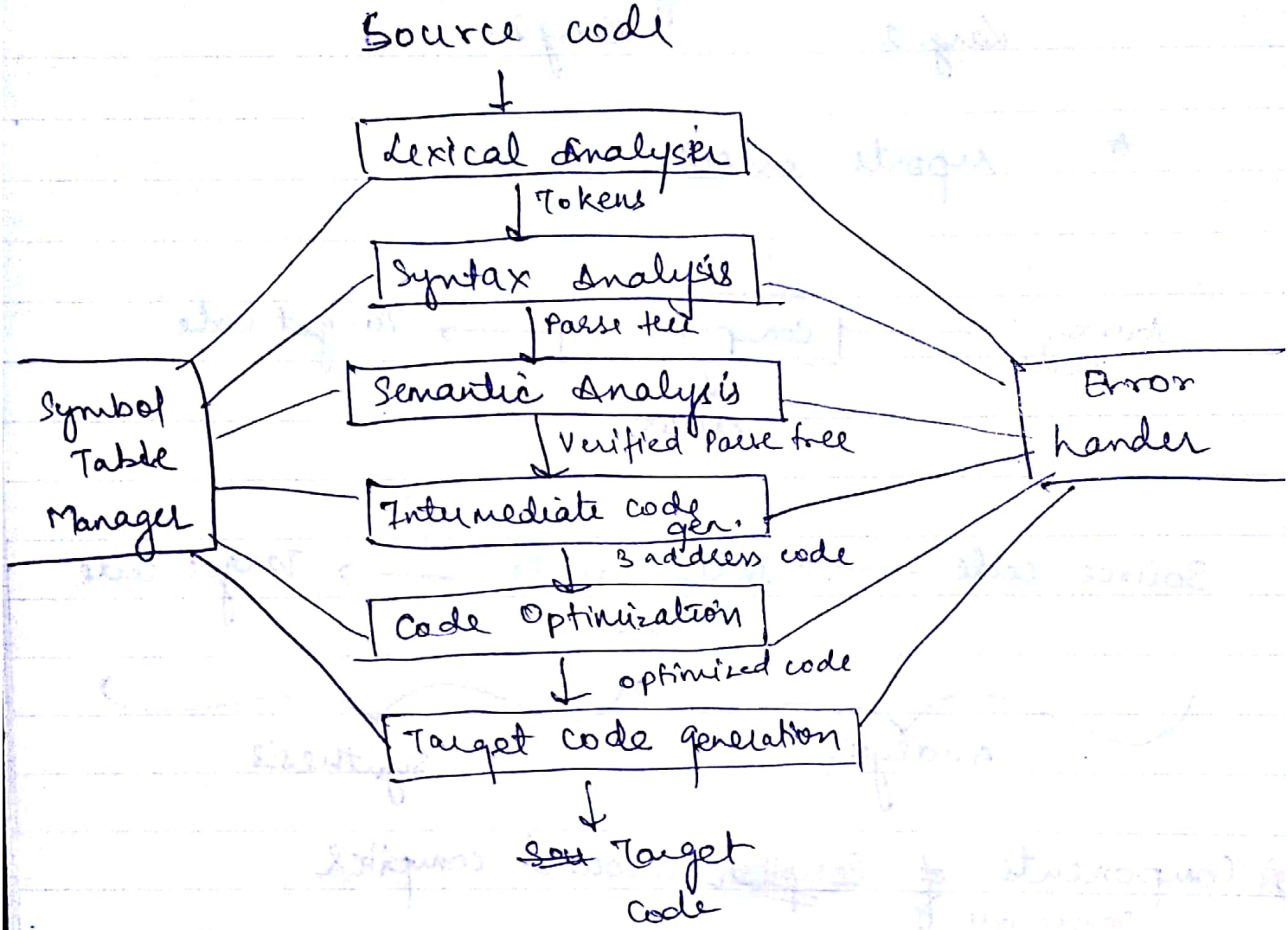
Source code → Intermediate Code → Target code

Analysis | Synthesis

☆ Components ~~of~~ ~~Compiler~~ around compiler

Source code
↓
Preprocessor → macro expansion & file inclusion
↓ modified
Compiler
↓ Target Assembly code
Assembler
↓ Relocatable m/c code
Loader/Linker
↓
Absolute m/c code

Phases of compiler



Symbol Table :- DS that stores various identifiers & their attributes.

Lexical Analysis

Source code ⟶ tokens

valid words of a language

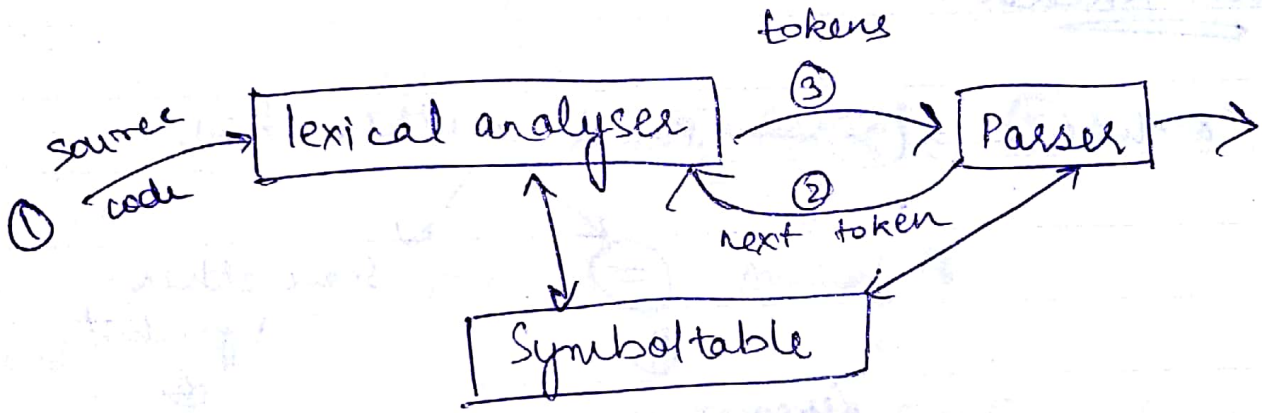
Lexeme :- Instance of a token

For eg :-

operators \\token

+, -, * \\lexeme

Patteen :- the sequence that is used to identify the type of token.



removes comments, whitespaces etc.

file inclusion & macro expansion

Displays errors.

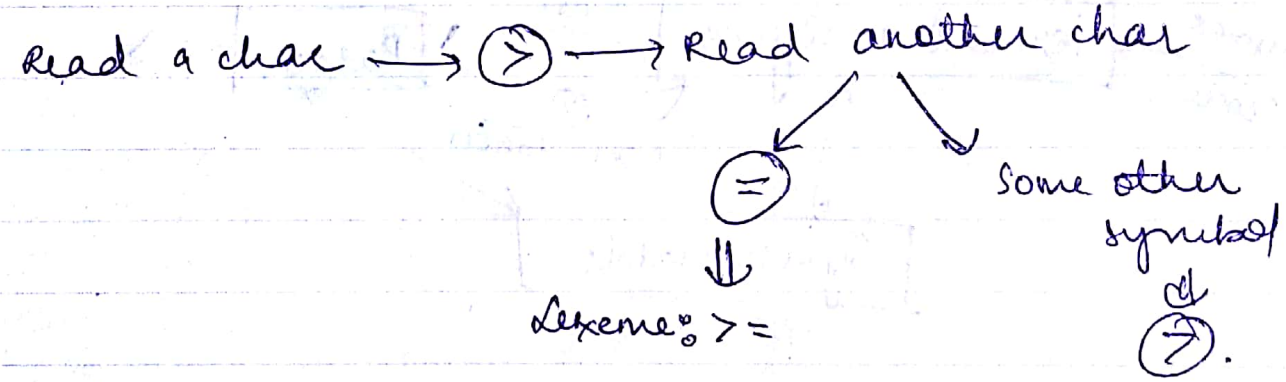
tokens → identifiers, operators, constant

Lexical Errors :- if a token is not matched.

→ delete the extra character.
→ insert a character
→ transpose two adjacent char.
→ replace

* The pointer to symbol table → attribute of token.

Look Ahead :- a >= b



Cross Compiler
A compiler that runs one one computer but produces obj code for a different computer.

Bootstrapping : Process by which simple lang. is used to compile more complicated lang and so on.

Quick and dirty compiler

Quick :- compiles code as fast as possible
mostly time ~~inefficient~~ inefficient.
checks functionality & correctness of code

Dirty :- efficient
time & space efficient
compiles with precision
optimized.

Shift Reduce Parsing

Four actions :-

① Shift → next symbol onto stack
② Reduce → pops
③ Accept → successful
④ Error

$E \rightarrow E+E$, $E \rightarrow E*E$, $E \rightarrow id$

Right form	Handle	Reduce Prod^n
$id_1 + id_2 * id_3$	id_1	$E \rightarrow id$
$E + id_2 * id_3$	id_2	$E \rightarrow id$
$E + E * id_3$	id_3	$E \rightarrow id$
$E + E * E$	$E * E$	$E \rightarrow E*E$
$E + E$	$E + E$	$E \rightarrow E+E$
E		

Reverse :-

Stack	I/p	Action
$	$id_1 + id_2 * id_3$	shift
$ id_1	$+ id_2 * id_3$	reduce $E \to id$
$ E	$+ id_2 * id_3$	shift
$ $E+$	$id_2 * id_3$	shift
$ $E+id_2$	$* id_3$	reduce $E \to id$
$ $E+E$	$* id_3$	shift
$ $E+E*$	id_3	shift
$ $E+E*id_3$		reduce $E \to id$
$ $E+E*E$		" $E \to E*E$
$ $E+E$		" $E \to E+E$
$ E		

Operator precedence Parsing

Operator Grammar → no null productions
→ no adjacent non-terminals on LHS.

eg :- $S \to ABa$ ✗

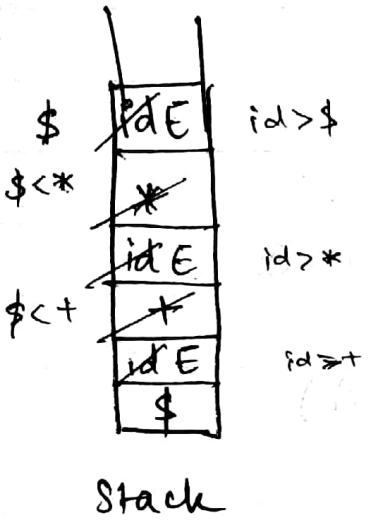
$S \to bAaC$ ✓

$S \to \epsilon$ ✗

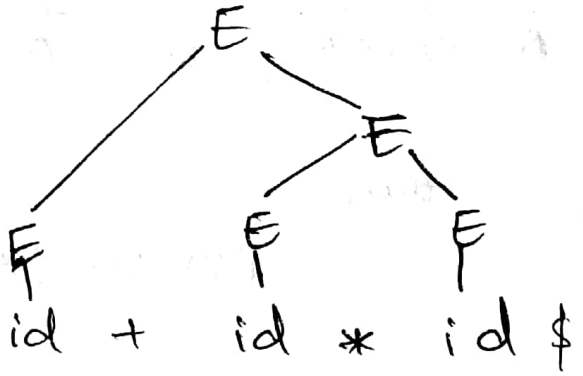
Eg. $E \rightarrow E+E / E*E / id$ id+id*id$

	id	+	*	$
id	-	>	>	>
+	<	-	<	>
*	<	>	-	>
$	<	<	<	-

Priority wise ← sign



Stack



First ()

① 1st terminal on LHS

eg:- $S \rightarrow aA$

First (S) = a

$S \rightarrow Aab$

$A \rightarrow cd$

First (S) = First (A) = c

$S \rightarrow C$

$C \rightarrow \epsilon$

First (S) = ϵ

$S \rightarrow \epsilon ab$

First (S) = a

Follow ()

- to the immediate right

1) For every start symbol
   Follow(S) = {$}

2) if $A \to \alpha B \beta$
   Follow(B) = First (β) except ϵ

3) if $A \to \alpha B$ or $A \to \alpha B \beta$ where FIRST(β) = ϵ

then

FOLLOW(B) = FOLLOW(A)

eg :-

$E \to TE'$
$E' \to +TE' / \epsilon$
$T \to FT'$
$T' \to *FT' / \epsilon$
$F \to (E) / id$

FOLLOW(E) = {$,)}
(E') = {$,)}
(T) = {+, $,)}
(T') = {+, $,)}
(F) = {*, +, $,)}

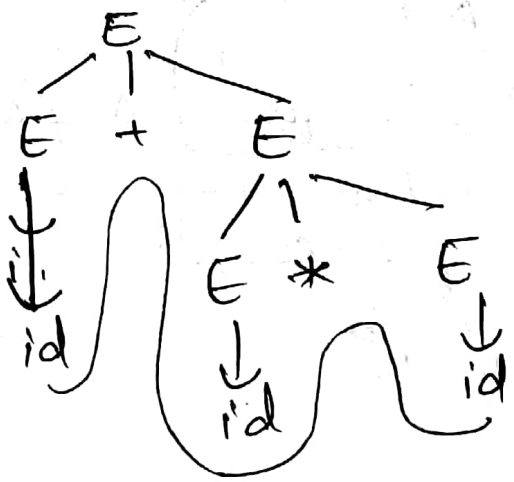
Top-down Parsing

→ deducing the i/p string ~~from to start sym.~~ from start symbol.

→ Root to leaves.

→ LMD

$E \rightarrow E+E/E*E/id$



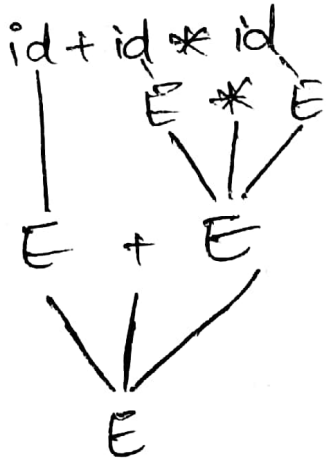
Bottom-up parsing

→ reducing the i/p string to ~~string~~ start symbol

→ leaves to root

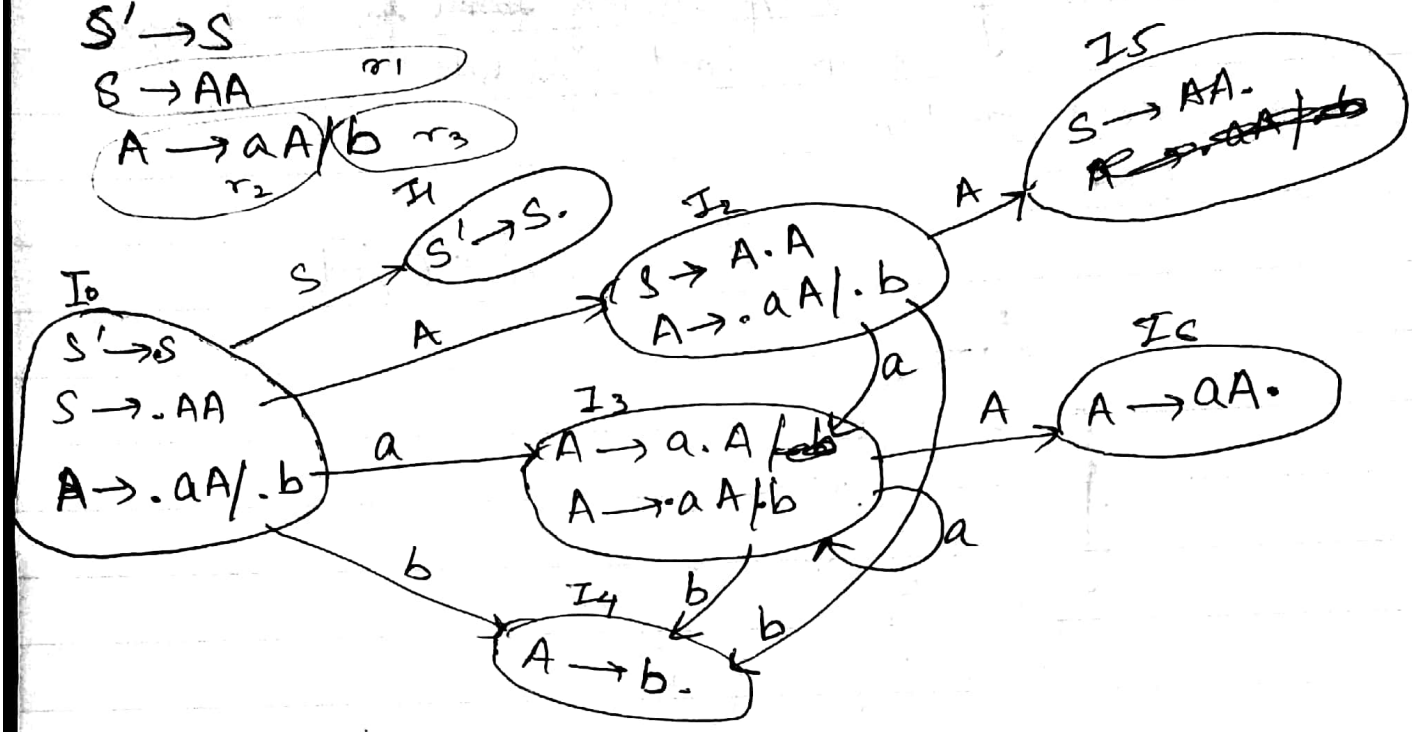
→ shift reduce parsing

→ LR & RMD in reverse.



LR(0)

Keep moving the dot



LR(0) Table

state	action a	action b	action $	Goto A	Goto S
0	S_3	S_4		2	1
1			accept		
2	S_3	S_4		5	
3	S_3	S_4		6	
4	r_3	r_3	r_3		
5	r_1	r_1	r_1		
6	r_2	r_2	r_2		

$S \to AA$ be r_1

$A \to aA$ be r_2

$S \to b$ be r_3

final states

I_4, I_5 & I_6

$I_4 \to r_3$, $I_5 \to r_1$, $I_6 \to r_2$

S_3 ↦ I_3, for $S' \to S.$ use accept,

LL(1)

	First	Follow
$E \to TE'$	{id, (}	{$,)}
$E' \to +TE' / \epsilon$	{+, ϵ}	{$,)}
$T \to FT'$	{id, (}	{+,), $}
$T' \to *FT' / \epsilon$	{*, ϵ}	{+,), $}
$F \to id / (E)$	{id, (}	{*, +,), $}

LL(1) parsing table

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'	~~E→ϵ~~	$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

jab first mein ϵ aaye, follow wali places par ϵ wali production likho, otherwise first wali pe

SLR(1) → powerful

Table of LR(0)

	a	b	$	A	S
0	S_3	S_4		2	1
1			accept		
2	S_3	S_4		5	
3	S_3	S_4		6	
4	r_3	r_3	r_3		
5	r_1	r_1	r_1		
6	r_2	r_2	r_2		

$S' \to S$

$S \to AA$ r_1

$A \to aA / b$ r_2 r_3

} in SLR(1), the ~~state~~ reduce moves are applied to follow of LHS.

	a	b	$	A	S
0	S_3	S_4		2	1
1			accept		
2	S_3	S_4		5	
3	S_3	S_4		6	
4	r_3	r_3	r_3		
5			r_1		
6	r_2	r_2	r_2		

State 4 (A → b.)

find follow(A) = first(A) & follow(S)

State 5 (S → AA.)

State 6 (A → aA.)

FOLLOW(A) for state 4

FIRST(A) = a and b

FOLLOW(S) → $

FOLLOW(S) for state 5

FOLLOW(S) = $

FOLLOW(A) for state 6

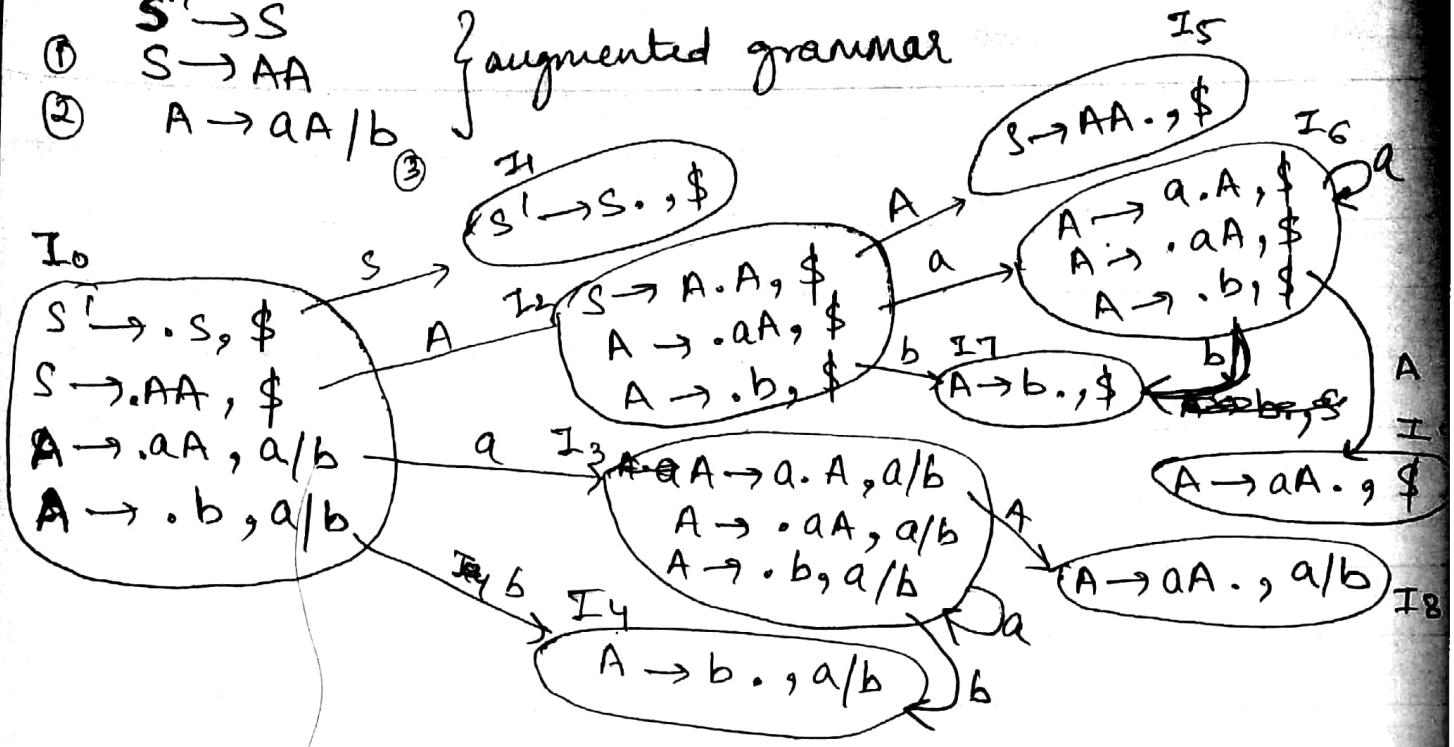
FIRST(A) = a, b

FOLLOW(S) = $

LALR(1) & CLR(1)

LR(0) + lookahead

① $S' \to S$
② $S \to AA$
③ $A \to aA / b$ } augmented grammar



CLR(1) parsing Table

	a	b	$	S	A
0	S_3	S_4		1	2
1	~~S_6~~	~~S_7~~			
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

LALR(1) parsing Table

	a	b	$	S	A
0	S_{36}	S_{47}		1	2
1					
2	~~S_6~~ S_{36}	S_{47}			5
36	S_{36}	S_{47}			89
47	r_3	r_3	r_3		
5			r_1		
89	r_2	r_2	r_2		

r at lookaheads

$[I_6, I_9]$, $[I_4, I_7]$, $[I_3\ I_6]$ only have diff. lookaheads

∴ merge them